

ECx00G&EC600U&EG800G Series

QuecOpen(SDK) Serial Port API Reference Manual

LTE Standard Module Series

Version: 1.0

Date: 2023-09-21

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

History

Revision	Date	Author	Description
-	2023-08-03	Sum LI	Creation of the document
1.0	2023-09-21	Sum LI	First official release

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules	6
2 Serial Port API	7
2.1. Header File	7
2.2. API Overview.....	7
2.3. API Description.....	8
2.3.1. ql_uart_set_dcbconfig.....	8
2.3.1.1. ql_uart_port_number_e	8
2.3.1.2. ql_uart_config_s	9
2.3.1.3. ql_uart_errcode_e	10
2.3.1.4. ql_uart_baud_e.....	11
2.3.1.5. ql_uart_databit_e	13
2.3.1.6. ql_uart_stopbit_e	13
2.3.1.7. ql_uart_parityit_e	13
2.3.1.8. ql_uart_flowctrl_e	14
2.3.2. ql_uart_get_dcbconfig.....	14
2.3.3. ql_uart_open	15
2.3.4. ql_uart_close	15
2.3.5. ql_uart_write.....	16
2.3.6. ql_uart_read	16
2.3.7. ql_uart_register_cb	17
2.3.7.1. ql_uart_callback.....	17
3 Examples	19
3.1. Development Example	19
3.2. Feature Debugging.....	20
4 Appendix References	22

Table Index

Table 1: Applicable Modules.....	6
Table 2: API Overview	7
Table 3: Related Documents.....	22
Table 4: Terms and Abbreviations	22

1 Introduction

Quectel ECx00G family, EC600U series and EG800G series modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces the serial port API functions, related examples and feature debugging of ECx00G family, EC600U series and EG800G series modules. The modules support the software to configure 3 or 6 physical serial ports and 3 USB virtual serial ports. The RX of the physical serial port uses the DMA mode by default, and the TX uses the FIFO mode by default.

NOTE

EC600U series module supports the software to configure 3 physical serial ports; ECx00G family and EG800G series modules support the software to configure 6 physical serial ports.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC600U Series
ECx00G	EC200G-CN
	EC600G Series
	EC800G-CN
	EG800G Series

2 Serial Port API

2.1. Header File

ql_uart.h, the header file of serial port API, is in the *components\ql-kernel\in* directory in SDK. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_uart_set_dcbconfig()</i>	Configures the attributes of serial port.
<i>ql_uart_get_dcbconfig()</i>	Gets the attribute configuration of serial port.
<i>ql_uart_open()</i>	Opens the specified serial port.
<i>ql_uart_close()</i>	Closes the specified serial port.
<i>ql_uart_write()</i>	Writes data to the specified serial port.
<i>ql_uart_read()</i>	Reads the data from the specified serial port.
<i>ql_uart_register_cb()</i>	Registers the callback function for serial port events.

2.3. API Description

2.3.1. ql_uart_set_dcbconfig

This function configures the attributes of serial port. The parameters configured by this function take effect after calling `ql_uart_open()` to reopen the serial port.

- **Prototype**

```
ql_uart_errcode_e ql_uart_set_dcbconfig(ql_uart_port_number_e port, ql_uart_config_s *dcb)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

dcb:

[In] Serial port attribute configuration. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.3** for details.

2.3.1.1. ql_uart_port_number_e

Currently, 3 or 6 physical serial ports and 3 USB virtual serial ports are supported by the module. The enumeration of the serial port number is defined as follows:

```
typedef enum
{
    QL_PORT_NONE = -1,
    QL_UART_PORT_1,
    QL_UART_PORT_2,
    QL_UART_PORT_3,
    QL_UART_PORT_4,
    QL_UART_PORT_5,
    QL_UART_PORT_6,
    QL_USB_PORT_AT,
    QL_USB_PORT_MODEM,
    QL_USB_PORT_NMEA,
    QL_PORT_MAX,
}ql_uart_port_number_e
```

● Member

Member	Description
QL_UART_PORT_1	UART1
QL_UART_PORT_2	UART2
QL_UART_PORT_3	UART3
QL_UART_PORT_4	UART4
QL_UART_PORT_5	UART5
QL_UART_PORT_6	UART6
QL_USB_PORT_AT	USB AT port
QL_USB_PORT_MODEM	USB Modem port
QL_USB_PORT_NMEA	USB NMEA port

NOTE

1. EC600U series module supports the software to configure UART1–UART3. ECx00G family and EG800G series modules support the software to configure UART1–UART6.
2. By default, ECx00G family and EG800G series modules use UART2 as the UART for CP log output and UART4 as the debug UART for AP log output, see **document [3]** for details.

2.3.1.2. ql_uart_config_s

The structure of serial port attribute configuration is defined as follows:

```
typedef struct
{
    ql_uart_baud_e baudrate;
    ql_uart_databit_e data_bit;
    ql_uart_stopbit_e stop_bit;
    ql_uart_paritybit_e parity_bit;
    ql_uart_flowctrl_e flow_ctrl;
}ql_uart_config_s
```

● Parameter

Type	Parameter	Description
<i>ql_uart_baud_e</i>	<i>baudrate</i>	Baud rate. The default value is 115200 bps. See Chapter 2.3.1.4 for details.
<i>ql_uart_databit_e</i>	<i>data_bit</i>	Data bit. The default value is 8 bits. See Chapter 2.3.1.5 for details.
<i>ql_uart_stopbit_e</i>	<i>stop_bit</i>	Stop bit. The default value is 1 bit. See Chapter 2.3.1.6 for details.
<i>ql_uart_paritybit_e</i>	<i>parity_bit</i>	Parity bit. There is no parity by default. See Chapter 2.3.1.7 for details.
<i>ql_uart_flowctrl_e</i>	<i>flow_ctrl</i>	Flow control. It's disabled by default. See Chapter 2.3.1.8 for details.

2.3.1.3. ql_uart_errcode_e

The error codes of the serial port indicate whether the API execution is successful or not. If the function execution is failed, the reason for error is returned. The enumeration of error codes is defined as follows:

```
typedef enum
{
    QL_UART_SUCCESS                = 0,
    QL_UART_EXECUTE_ERR            = 1|QL_UART_ERRCODE_BASE,
    QL_UART_MEM_ADDR_NULL_ERR,
    QL_UART_INVALID_PARAM_ERR,
    QL_UART_OPEN_REPEAT_ERR,
    QL_UART_NOT_OPEN_ERR          = 5|QL_UART_ERRCODE_BASE,
} ql_uart_errcode_e
```

● Member

Member	Description
<i>QL_UART_SUCCESS</i>	Successful execution.
<i>QL_UART_EXECUTE_ERR</i>	Failed execution.
<i>QL_UART_MEM_ADDR_NULL_ERR</i>	The parameter address is NULL.
<i>QL_UART_INVALID_PARAM_ERR</i>	Invalid parameter.
<i>QL_UART_OPEN_REPEAT_ERR</i>	The serial port is opened repeatedly.
<i>QL_UART_NOT_OPEN_ERR</i>	The serial port is not opened.

2.3.1.4. ql_uart_baud_e

The enumeration of baud rate is defined as follows:

```
typedef enum
{
    QL_UART_BAUD_AUTO      = 0,
    QL_UART_BAUD_1200      = 1200,
    QL_UART_BAUD_2400      = 2400,
    QL_UART_BAUD_4800      = 4800,
    QL_UART_BAUD_9600      = 9600,
    QL_UART_BAUD_14400     = 14400,
    QL_UART_BAUD_19200     = 19200,
    QL_UART_BAUD_28800     = 28800,
    QL_UART_BAUD_33600     = 33600,
    QL_UART_BAUD_38400     = 38400,
    QL_UART_BAUD_57600     = 57600,
    QL_UART_BAUD_115200    = 115200,
    QL_UART_BAUD_187500    = 187500,
    QL_UART_BAUD_230400    = 230400,
    QL_UART_BAUD_250000    = 250000,
    QL_UART_BAUD_460800    = 460800,
    QL_UART_BAUD_500000    = 500000,
    QL_UART_BAUD_921600    = 921600,
    QL_UART_BAUD_1000000   = 1000000,
    QL_UART_BAUD_1843200   = 1843200,
    QL_UART_BAUD_2000000   = 2000000,
    QL_UART_BAUD_2100000   = 2100000,
    QL_UART_BAUD_3686400   = 3686400,
    QL_UART_BAUD_4000000   = 4000000,
    QL_UART_BAUD_4468750   = 4468750
}ql_uart_baud_e;
```

● Member

Member	Description
<i>QL_UART_BAUD_AUTO</i>	Adaptive baud rate.
<i>QL_UART_BAUD_1200</i>	Baud rate is 1200 bps.
<i>QL_UART_BAUD_2400</i>	Baud rate is 2400 bps.
<i>QL_UART_BAUD_4800</i>	Baud rate is 4800 bps.
<i>QL_UART_BAUD_9600</i>	Baud rate is 9600 bps.

<i>QL_UART_BAUD_14400</i>	Baud rate is 14400 bps.
<i>QL_UART_BAUD_19200</i>	Baud rate is 19200 bps.
<i>QL_UART_BAUD_28800</i>	Baud rate is 28800 bps.
<i>QL_UART_BAUD_33600</i>	Baud rate is 33600 bps.
<i>QL_UART_BAUD_38400</i>	Baud rate is 38400 bps.
<i>QL_UART_BAUD_57600</i>	Baud rate is 57600 bps.
<i>QL_UART_BAUD_115200</i>	Baud rate is 115200 bps.
<i>QL_UART_BAUD_187500</i>	Baud rate is 187500 bps.
<i>QL_UART_BAUD_230400</i>	Baud rate is 230400 bps.
<i>QL_UART_BAUD_250000</i>	Baud rate is 250000 bps.
<i>QL_UART_BAUD_460800</i>	Baud rate is 460800 bps.
<i>QL_UART_BAUD_500000</i>	Baud rate is 500000 bps.
<i>QL_UART_BAUD_921600</i>	Baud rate is 921600 bps.
<i>QL_UART_BAUD_1000000</i>	Baud rate is 1000000 bps.
<i>QL_UART_BAUD_1843200</i>	Baud rate is 1843200 bps.
<i>QL_UART_BAUD_2000000</i>	Baud rate is 2000000 bps.
<i>QL_UART_BAUD_2100000</i>	Baud rate is 2100000 bps.
<i>QL_UART_BAUD_3686400</i>	Baud rate is 3686400 bps.
<i>QL_UART_BAUD_4000000</i>	Baud rate is 4000000 bps.
<i>QL_UART_BAUD_4468750</i>	Baud rate is 4468750 bps.

NOTE

For EC600U series module, UART1 currently support a maximum baud rate of 2100000 bps, but do not support a maximum baud rate of 250000 bps, 500000 bps or 2000000 bps. UART2 and UART3 support a maximum baud rate of 4468750 bps, but do not support a baud rate of 250000 bps or 500000 bps. For ECx00G family and EG800G series modules, the UARTs support a maximum baud rate of 2000000 bps.

2.3.1.5. ql_uart_databit_e

The enumeration of data bit is defined as follows:

```
typedef enum
{
    QL_UART_DATABIT_7 = 7,
    QL_UART_DATABIT_8 = 8,
}ql_uart_databit_e
```

● Member

Member	Description
QL_UART_DATABIT_7	7 data bits (Not supported currently.)
QL_UART_DATABIT_8	8 data bits.

2.3.1.6. ql_uart_stopbit_e

The enumeration of stop bit is defined as follows:

```
typedef enum
{
    QL_UART_STOP_1 = 1,
    QL_UART_STOP_2 = 2,
}ql_uart_stopbit_e
```

● Member

Member	Description
QL_UART_STOP_1	1 stop bit.
QL_UART_STOP_2	2 stop bits.

2.3.1.7. ql_uart_parityit_e

The enumeration of parity bit is defined as follows:

```
typedef enum
{
    QL_UART_PARITY_NONE,
    QL_UART_PARITY_ODD,
```

```

    QL_UART_PARITY_EVEN,
}ql_uart_paritybit_e

```

● Member

Member	Description
<i>QL_UART_PARITY_NONE</i>	No parity bit.
<i>QL_UART_PARITY_ODD</i>	Enable odd parity.
<i>QL_UART_PARITY_EVEN</i>	Enable even parity.

2.3.1.8. ql_uart_flowctrl_e

The enumeration of flow control is defined as follows:

```

typedef enum
{
    QL_FC_NONE = 0,
    QL_FC_HW,
}ql_uart_flowctrl_e

```

● Member

Member	Description
<i>QL_FC_NONE</i>	Disable hardware flow control.
<i>QL_FC_HW</i>	Enable hardware flow control.

2.3.2. ql_uart_get_dcbconfig

This function gets the attribute configuration of serial port.

● Prototype

```

ql_uart_errcode_e ql_uart_get_dcbconfig(ql_uart_port_number_e port, ql_uart_config_s *dcb)

```

● Parameter

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

dcb:

[Out] Attribute configuration of serial port. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.3** for details.

2.3.3. ql_uart_open

This function opens the specified serial port.

- **Prototype**

```
ql_uart_errcode_e ql_uart_open(ql_uart_port_number_e port)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.3** for details.

2.3.4. ql_uart_close

This function closes the specified serial port.

- **Prototype**

```
ql_uart_errcode_e ql_uart_close(ql_uart_port_number_e port)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.3** for details.

2.3.5. ql_uart_write

This function writes data to the specified serial port.

- **Prototype**

```
int ql_uart_write(ql_uart_port_number_e port, unsigned char *data, unsigned int data_len)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

data:

[In] Data to be written.

data_len:

[In] Length of the data to be written. Unit: byte.

- **Return Value**

Negative numbers Failed execution. See **Chapter 2.3.1.3** for error codes.

Other values Length of the actual written data.

2.3.6. ql_uart_read

This function reads the data from the specified serial port.

- **Prototype**

```
int ql_uart_read(ql_uart_port_number_e port, unsigned char *data, unsigned int data_len)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

data:

[Out] The data read.

data_len:

[In] Length of the read data. Unit: byte.

- **Return Value**

Negative numbers Failed execution. See error codes in **Chapter 2.3.1.3** for details.
Other values Length of the actual read data.

2.3.7. ql_uart_register_cb

This function registers the callback function for serial port events.

- **Prototype**

```
ql_uart_errcode_e ql_uart_register_cb(ql_uart_port_number_e port, ql_uart_callback uart_cb)
```

- **Parameter**

port:

[In] Serial port number. See **Chapter 2.3.1.1** for details.

uart_cb:

[In] The callback function to be registered. If the function is set to NULL, the system is in polling pattern.
See **Chapter 2.3.7.1** for details.

- **Return Value**

See **Chapter 2.3.1.3** for details.

2.3.7.1. ql_uart_callback

This function is the callback function of serial port events, used to notify the client to read the data.

- **Prototype**

```
typedef void (*ql_uart_callback)(uint32 ind_type, ql_uart_port_number_e port, uint32 size)
```

- **Parameter**

ind_type:

[In] The types of serial port event. The events that can be handled on the App side include:

<code>QUEC_UART_RX_RECV_DATA_IND</code>	The event of receiving data.
<code>QUEC_UART_RX_OVERFLOW_IND</code>	The event of buffer overflow after receiving data.
<code>QUEC_UART_TX_FIFO_COMPLETE_IND</code>	The event of finishing transmitting in FIFO mode.

port:

[In] The number of the serial port from which an event is sourced. See **Chapter 2.3.1.1** for details.

size:

[In] The size of data received by the serial port. Unit: byte.

- **Return Value**

None

3 Examples

This chapter introduces how to use the above API functions for serial port development and simple feature debugging on the App side. In the example, UART 2 is used for test by default.

3.1. Development Example

The example file *components\ql-application\peripheral\uart_demo.c* of serial port API, is provided in the SDK codes of ECx00G family, EC600U series and EG800G series modules.

The file function is analyzed as follows:

- *ql_uart_app_init()* //Creates a task. This function needs to be called before enabling the example.
- *ql_uart_demo_thread()* //Executes the task. This function simply realizes the data configuring, opening, reading and writing of the serial port. The interrupt callback function reads the data by default.
- *ql_uart_notify_cb()* //The callback function for serial port events, which simply realizes the data reading. The received data can be printed through coolwatcher.

If you need to enable this example, call *ql_uart_app_init()* in *ql_init_demo_thread()*, as shown in the following figure:

```

static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);

    ql_gpio_app_init();
    ql_gpioint_app_init();
    ql_ledcfg_app_init();

#ifdef QL_APP_FEATURE_AUDIO
    ql_audio_app_init();
#endif
#ifdef QL_APP_FEATURE_LCD
    ql_lcd_app_init();
#endif
    ql_nw_app_init();
    //ql_datacall_app_init();
    ql_osi_demo_init();

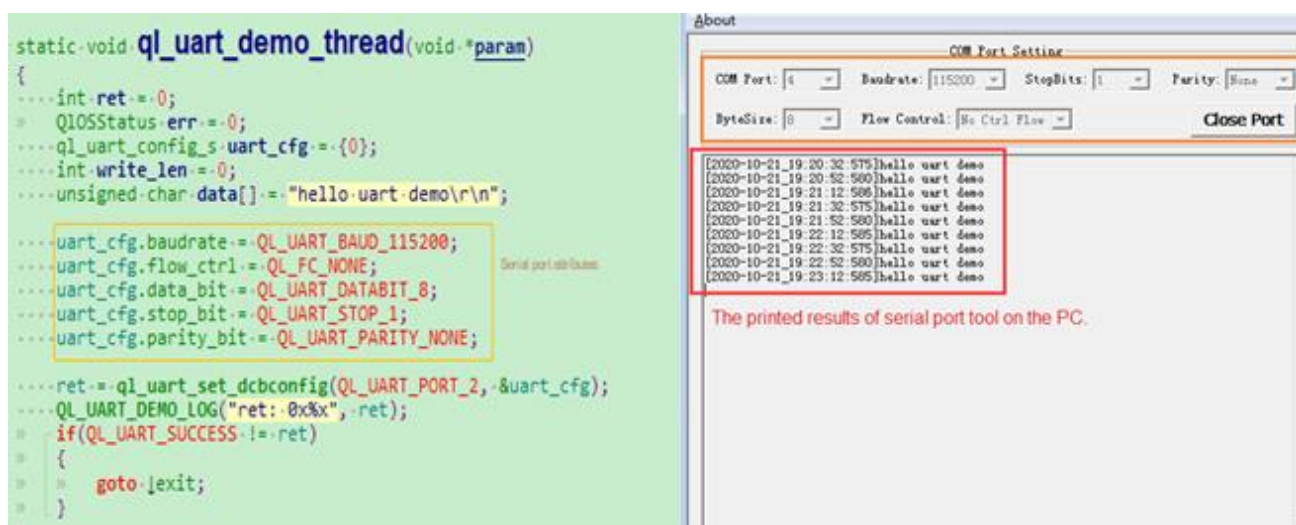
#ifdef QL_APP_FEATURE_FILE
    ql_fs_demo_init();
#endif

    //ql_dev_app_init();
    ql_adc_app_init();
    ql_uart_app_init();
#ifdef QL_APP_FEATURE_HTTP
    ql_http_app_init();
#endif
}
    
```

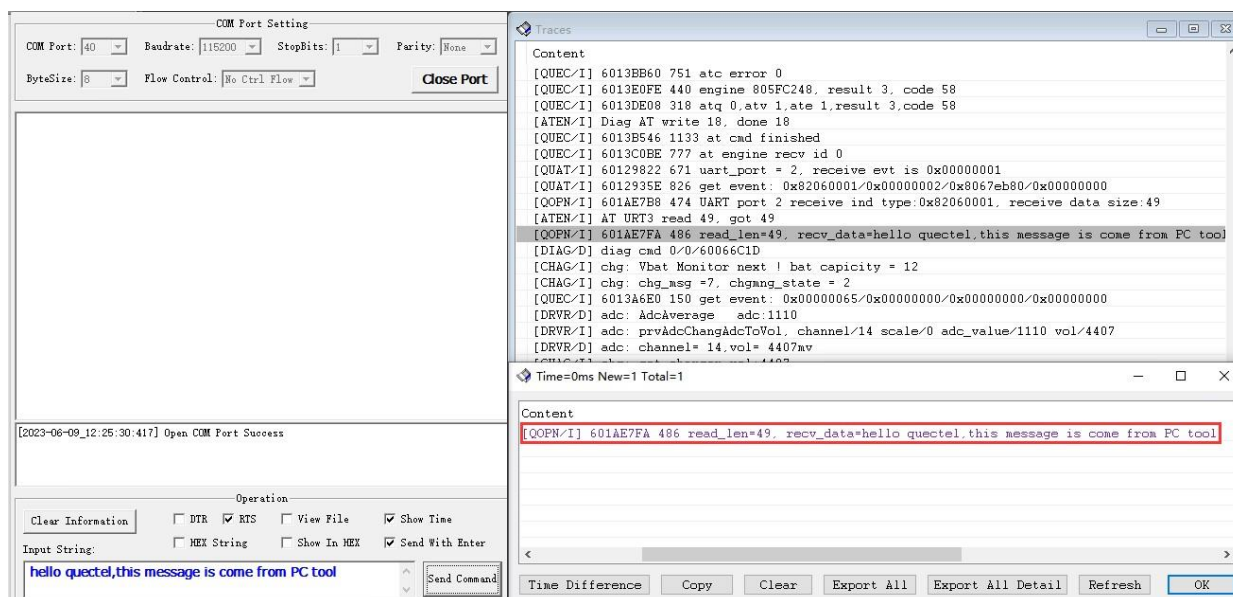
3.2. Feature Debugging

You can debug the serial port function of ECx00G family, EC600U series and EG800G series modules through LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to the PC with a USB cable. The USB AP Log Port or USB DIAG Port mainly displays the system debugging information. Connect the cooltools or ArmLogel tool, open the serial port tool (such as QCOM) on the PC and connect the serial port according to the serial port attributes set in the example. You can see that "hello uart demo" is received in the serial port tool of the PC as shown in the following figure:



Send the message "hello quectel, this message is come from PC tool" through the serial port tool. The serial port data, received successfully by the module can be printed with cooltools or ArmLogel tool as shown in the following figure:



NOTE

For EC600U series module, cooltools tool is used to capture logs through USB AP Log Port, see **document [2]** for details on log capture. For ECx00G family and EG800G series modules, ArmLogel tool is used to capture logs through USB DIAG Port, see **document [3]** for details on log capture.

4 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC600U_Series_QuecOpen(SDK)_Log_Capture_Guide
[3] Quectel_ECx00G&EG800G_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
App	Application
EVB	Evaluation Board
FIFO	First In First Out
IoT	Internet of Things
LTE	Long-Term Evolution
NMEA	NMEA (National Marine Electronics Association) 0183 Interface Standard
PC	Personal Computer
RTOS	Real-Time Operating System
RX	Receive
SDK	Software Development Kit
TX	Transmit
DMA	Direct Memory Access